Guia Prático do Aluno — Aula 7

Empacotamento, Conteinerização e Orquestração com Spring Boot 4.1.x, PostgreSQL e Docker Compose

Este documento é o seu guia prático oficial de acompanhamento para a **Aula 7: Empacotamento, Containers e Docker Compose**. Siga os laboratórios em sequência (Lab 0 ao Lab 4) conforme apresentados no material de apoio e projecções do instrutor.

1. Configuração do Terminal e Pré-Requisitos

> 🛠 **Padronização do Terminal:** No Windows, utilize sempre o terminal Git Bash integrado no IntelliJ IDEA (Settings > Tools > Terminal > Shell path: bash.exe). No macOS ou Linux, utilize o Terminal nativo (Zsh/Bash). Isso garante paridade total nos comandos executados.

Antes de iniciar os laboratórios, confirme que as seguintes portas estão livres na sua máquina local:
- **Porta 5432:** Reservada para o container do banco de dados PostgreSQL.
- **Porta 8081:** Reservada para o container da API MyMovies (evita conflitos com a 8080 nativa).
- **Porta 8080:** Utilizada apenas durante a execução standalone local no Lab 0.

LAB 3: Orquestração Multi-Container com Docker Compose

Objetivo: Integrar a API Java Spring Boot e o banco de dados PostgreSQL na mesma rede virtual conteinerizada com injeção de configurações externas.

Passo 3.1: Criar o Perfil application-postgres.properties

Crie o arquivo src/main/resources/application-postgres.properties no projeto:

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${POSTGRES_USER}
spring.datasource.password=${POSTGRES_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.sql.init.mode=never
spring.h2.console.enabled=false
```

Passo 3.2: Alterar o pom.xml

Incluir a dependência do postgres

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Passo 3.3: Atualizar o compose.yaml (Serviço api)

Atualize o compose.yaml incluindo a definição do serviço api integrado ao db:

```
name: mymovies-aula7

services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: mymovies
      POSTGRES_USER: ${POSTGRES_USER:-mymovies_lab}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-mymovies_pass}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -h 127.0.0.1 -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

  api:
    build: .
    image: my-movies:aula7
    env_file: .env.local
    environment:
      SPRING_PROFILES_ACTIVE: postgres
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mymovies
      POSTGRES_USER: ${POSTGRES_USER:-mymovies_lab}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-mymovies_pass}
    ports:
      - "127.0.0.1:8081:8080"
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

Passo 3.4: Criar o Ficheiro de Segredos .env.local

Crie o arquivo .env.local na raiz do projeto (fora do Git) contendo o token de integração do TMDB:

```
TMDB_TOKEN=seu_bearer_token_tmdb_aqui
```

Passo 3.5: Subir a Pilha Completa e Validar HTTP

Inicie toda a infraestrutura orquestrada e execute a verificação dos endpoints:

```
# Reconstruir o package para compor as alteraões
./mvnw package -DskipTests

# Desligar os containers ativos
docker compose down

# Subir a API e o Banco com build da imagem:
docker compose up -d --build

# Validar a saúde da API conectada ao PostgreSQL:
curl -i http://localhost:8081/actuator/health

# Retorno esperado: HTTP/1.1 200 OK {"status":"UP","components":{"db":{"status":"UP"...}}

# Se for usar o Bruno, lembre de alterar a porta para 8081
```

Status (DoD)	Critério de Sucesso do Laboratório
[ ] Concluído	Perfil application-postgres.properties e .env.local configurados
[ ] Concluído	Serviços api e db em execução e comunicando via hostname 'db'
[ ] Concluído	Endpoint http://localhost:8081/actuator/health retornando status UP

LAB 4: Persistência de Dados

Objetivo: Provar que os dados inseridos na base PostgreSQL sobrevivem ao reinício e destruição dos containers graças ao volume pgdata, finalizando com a publicação da tag Git.

Passo 4.1: Cadastrar Filme e Provar a Persistência

Cadastre um filme na API conteinerizada, destrua os containers com docker compose down e confirme a sobrevida dos dados:

```
# 1. Autenticar ou Cadastrar um Filme via Bruno / curl na porta 8081:
curl -X POST http://localhost:8081/api/movies \
  -H "Content-Type: application/json" \
  -d '{"title":"Inception","director":"Christopher Nolan","year":2010}'

# 2. Destruir os containers (sem remover o volume pgdata):
docker compose down

# 3. Subir novamente a infraestrutura:
docker compose up -d

# 4. Consultar os filmes cadastrados e comprovar que o registro permanece na base:
curl -i http://localhost:8081/api/movies

# Pode-se usar o Bruno ao invès do curl, mas lembrando da porta 8081
```

Status (DoD)	Critério de Sucesso do Laboratório
[ ] Concluído	Filme cadastrado e persistido no PostgreSQL via volume pgdata após docker compose down
[ ] Concluído	Consulta de filmes confirmando sobrevida dos dados após recriação do container